



DataBase Project

EJAR Machinery Rental



Team # 5			Phase # 3	
Students			Topic	
Name	ID	Section		
Ghala Alomran 445204572		41207	EJAR Machinery Rental	
Jory Alshaalan 445200359				
Leen Alharbi 445200423				
Tala Alshowiman 445202294				
Layan Alamry 445203042				



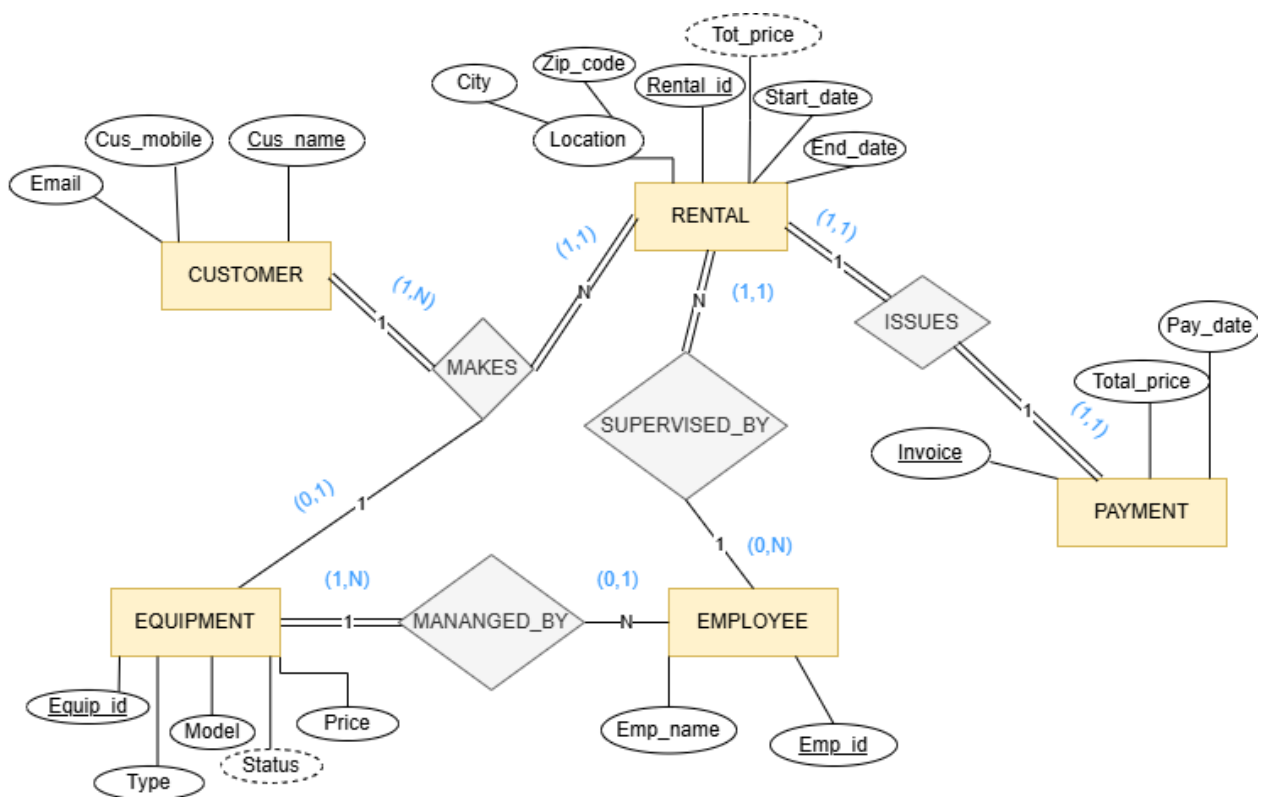
Description:

EJAR is a modern and user-friendly equipment rental service that helps companies across Saudi Arabia request and rent machinery such as generators, cranes, forklifts, excavators, concrete mixers, and other construction and industrial machinery with ease. Companies are the customer base who submit requests with the required equipment and a delivery site. Each rental includes exactly one equipment unit, records the rental details, and must always be supervised by an employee. Employees manage equipment, ensuring availability and status updates, and supervise rentals. Once a rental is confirmed, the system generates an invoice number, and payments are completed securely online.

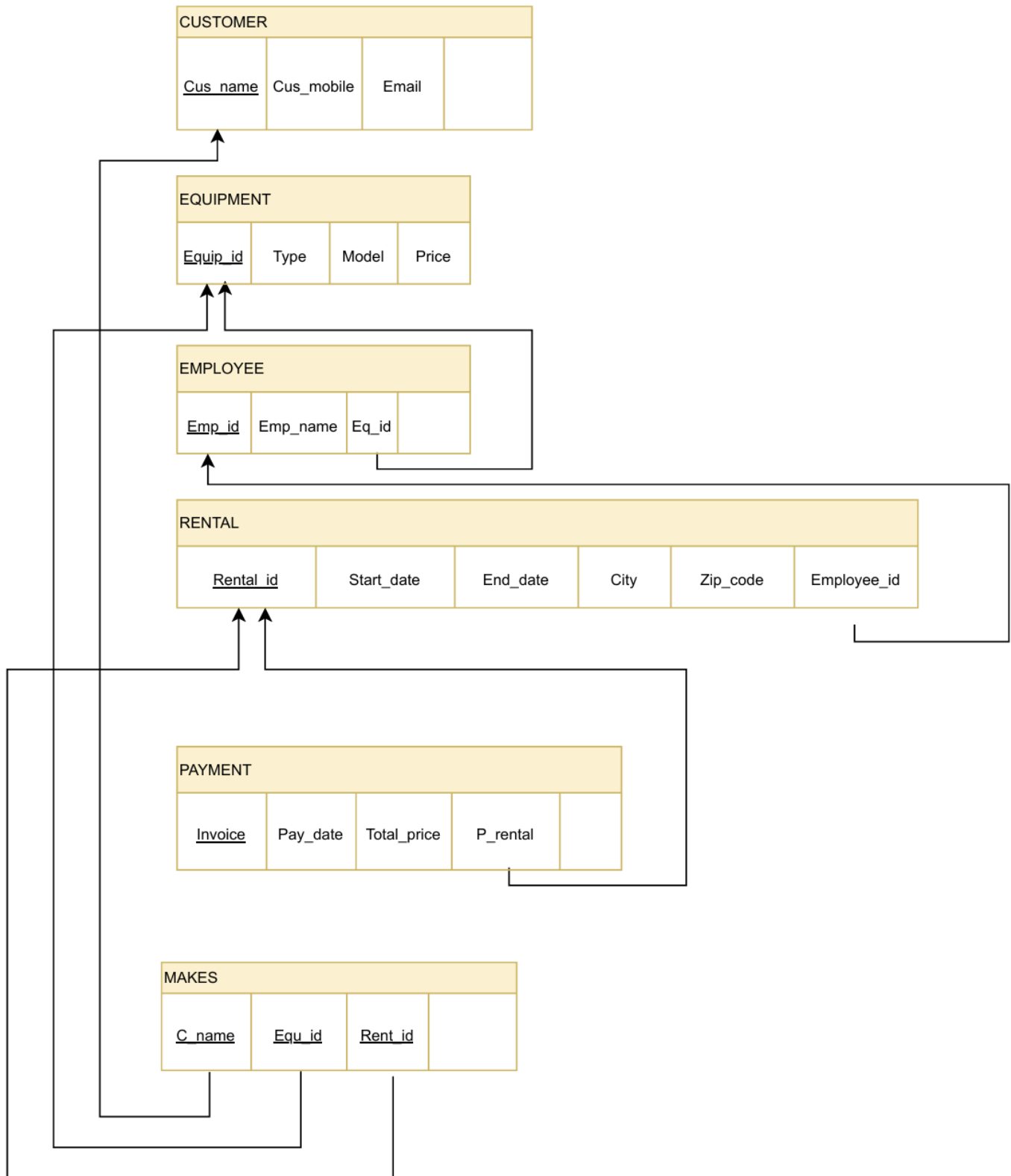
Requirements:

- The **Customer** is identified by a unique Company name, their email and mobile number. Customers represent the businesses that request equipment rentals through our EJAR system. To be registered in our system, a customer has to make at least one or as many rentals over time, and each rental must always belong to exactly one customer.
- The **Rental** is the contract created between a customer and EJAR. It is identified by a Rental ID and stores the rental start date, end date, total price, and location (city and zip code). Each rental must always include exactly one equipment unit, one customer, and be supervised by exactly one employee. Once a rental is approved, the customer must make the payment. Hence, a rental must issue exactly one payment.
- The **Equipment** is identified by its Serial number which is represented as the Equipment ID and stores details such as type, model, price, and current status (Available or Rented). The status of each equipment item changes automatically based on rentals. An equipment can be part of many rentals only over different rental periods, to avoid double booking.
- The **Employee** is identified by a unique Employee ID and stores the employee's name. Employees can manage an equipment, update its price or remove it -if not rented - when needed. Each employee can manage one equipment unit and can supervise many rentals.
- **Payment** records the payment date and total price (is determined automatically from the Equipment's daily price multiplied by the Rental's total days from start to end date). The invoice number uniquely identifies each payment. A payment is tied to exactly one rental.

ER Diagram:



Mapping :



Two Users View: Customer and Employee:

Customer View (responsibilities):

A customer's responsibilities in the system include registering their personal information to create an account, then immediately making their first rental since every customer must have at least one rental to exist in the system. After registration, the customer can update their personal details such as mobile number and email whenever needed. They can also view all information in their own rental records (along with their payment details). In addition, customers can browse information about available equipment, such as the type, model, and price. Each customer can only view their own data and can't access or view of other customers.

Cus_name Cus_mobile Email

Rental_id Start_date End_date City Zip_code Tot_price

Pay_date Total_Price Invoice

Equip_id Model Type Price

Employee View (responsibilities):

An employee is responsible for managing and maintaining the equipment and rental operations in the system. Employees can add new equipment to the database, and as an equipment manager can update its price or remove its record -if not rented- when needed. They also view and supervise rentals, such as overseeing the entire rental process, ensuring it runs smoothly and returned on time. In addition, employees can view payment information, including total amount, invoice, and payment date, to track customer transactions. Employees can view customers' contact info except for their contact mobile (unless Employee is a supervisor of the customer's rental). An employee can remove their record only if they are not supervising any rentals at the time.

Cus_name Cus_mobile Email

Rental_id Start_date End_date City Zip_code Tot_price

Pay_date Total_Price Invoice

Equip_id Model Type Status Price

Emp_id Emp_name



Only the employee who supervises that rental can view it

Employee Operations:

INSERT:

1. INSERT INTO EQUIPMENT
VALUES (110, 'Construction Machine', 'CAT 320D Excavator 2016', 750.00, 'Available');
2. INSERT INTO EMPLOYEE
VALUES (3, 'Sara Al-Qahtani');

DELETE:

1. DELETE FROM EQUIPMENT
WHERE Equip_id = 111 ;
2. DELETE FROM EMPLOYEE
WHERE Emp_id = 3 ;

UPDATE:

1. UPDATE EQUIPMENT
SET Price = 2500.00
WHERE Equip_id = 1;
2. UPDATE RENTAL
SET Employee_id = 2
WHERE Rental_id = 3001;

Retrieve One table:

1. Show all rentals that are supervised by **employee #2**, listing each rental's ID, start/end date, and the rental's city and ZIP code.

```
SELECT Rental_id, Start_date, End_date, City, Zip_code  
  
FROM RENTAL  
  
WHERE Employee_id = 2;
```

2. List every piece of equipment that costs **more than 3000**, including its ID, type, model, and price.

```
SELECT Equip_id, Type, Model, Price  
  
FROM EQUIPMENT  
  
WHERE Price > 3000;
```

Retrieve two tables:

1. For each rental, show the rental's ID, start and end date, together with the name of the employee who supervises it.

```
SELECT R.Rental_id, R.Start_date, R.End_date, E.Emp_name  
  
FROM RENTAL R, EMPLOYEE E  
  
WHERE E.Emp_id = R.Employee_id;
```

Retrieve three tables:

2. For each machinery type, show total sales starting from best sales to least sales.

```
SELECT Type, SUM(Total_price) AS Total_sales  
  
FROM ( ( EQUIPMENT JOIN MAKES ON Equipment_id = Equip_id ( JOIN RENTAL ON Rent_id = Rental_id  
 ) JOIN PAYMENT ON Rental_id = P_rental )  
  
GROUP BY Type  
  
ORDER BY Total_sales DESC;
```

Customer Operations:

INSERT:

1. INSERT INTO CUSTOMER

VALUES ('Perfect Target', 0556354410, 'perfecttarget@gmail.com');

2. INSERT INTO RENTAL

VALUES (2001, '2024-11-20', '2024-11-25', 'Riyadh', '11564', 3750.00, 110, 'Perfect Target', 2);

DELETE:

1. DELETE FROM MAKES

WHERE Rent_id = 2001;

AND C_name = 'Perfect Target';

2. DELETE FROM MAKES

WHERE Rent_id = 3001;

AND C_name = 'AlJazeera';

UPDATE:

1. UPDATE CUSTOMER

SET Email = 'aljazeera2025@gmail.com'

WHERE Cus_name = 'AlJazeera';

2. UPDATE CUSTOMER

SET Cus_mobile= 0555783221

WHERE Cus_name = 'AlJazeera';

Retrieve One table:

1. Customers view their own account information

```
SELECT Cus_name, Cus_mobile, Email  
FROM CUSTOMER  
WHERE Cus_name = 'Perfect Target';
```

2. Browse all available equipment in the system

```
SELECT Equip_id, Type, Model, Price  
FROM EQUIPMENT  
WHERE Status = 'Available';
```

Retrieve two tables:

Show all rentals that belong to the Perfect Target customer

```
SELECT Rental_id, Start_date, End_date, City, Zip_code, Tot_price  
FROM RENTAL, MAKES  
WHERE C_name = 'Perfect Target' AND Rent_id= Rental_id ;
```

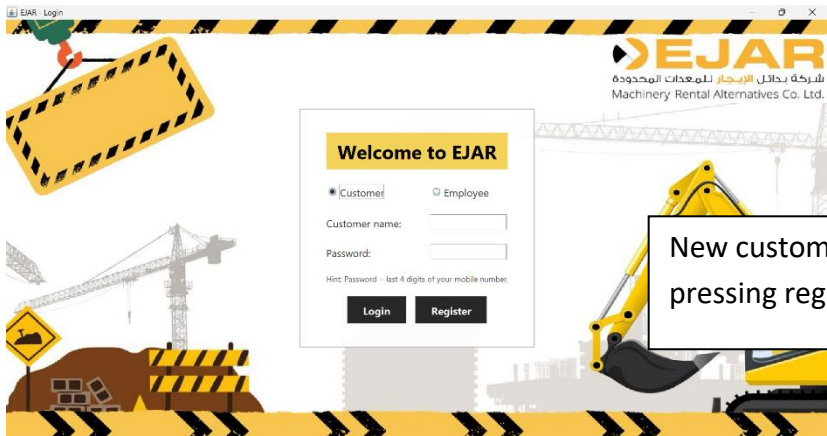
Retrieve three tables:

Show payment records for AlJazeera company from newest to oldest.

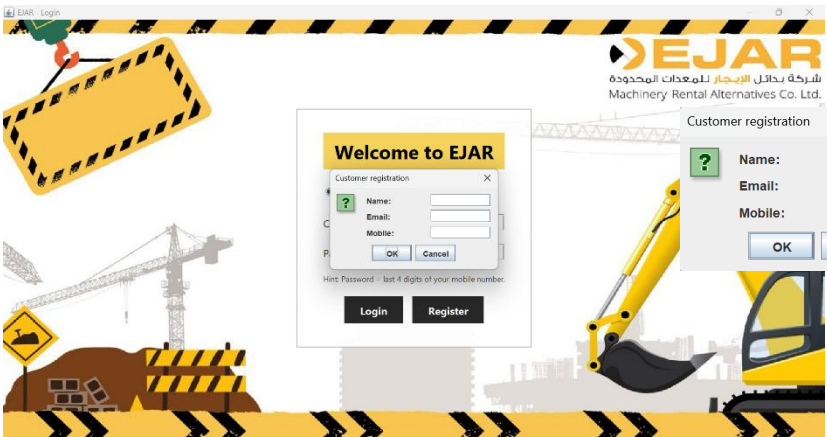
```
SELECT Invoice, Pay_date, Total_price  
FROM ( ( MAKES JOIN RENTAL ON Rent_id = Rental_id ) JOIN PAYMENT ON Rental_id = P_rental )  
WHERE C_name = 'AlJazeera'  
ORDER BY Pay_date DESC;
```

GUI :

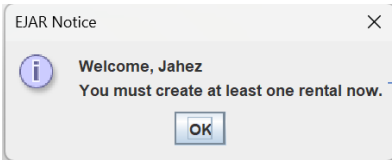
Customer Registration:



New customers can choose to register by pressing register button:



Example of registration



To ensure that customers are in the system with at least on rental, new customers cannot logout unless adding their first rental

After first rental:

EJAR – Customer Portal

Welcome, Jahez

Mobile: 0509871234 | Email: Jahez@gmail.com

Available Equipments

Equip ID	Type	Model	Price
122	Generator	Diesel 80KW	650.0
123	Concrete Pump	Putzmeister BSA 1407	2000.0
124	Mini Excavator	Kubota U55-4	700.0
125	Scissor Lift	JLG 3246ES	550.0

New Booking

Equipment ID:
Start Date:
End Date:
City:
Zip Code:

Confirm Booking

My Rentals

Cancel Booking

My Payments

Update My Info

Logout

New Booking

Equipment ID:
Start Date:
End Date:
City:
Zip Code:

Confirm Booking

New customers add rental by
Selecting available equipment from
the list then adding rental info:

After confirming first rental, logout
can be viewed and pressed.

EJAR – Customer Portal

Welcome, Jahez

Mobile: 0509871234 | Email: Jahez@gmail.com

Available Equipments

Equip ID	Type	Model	Price
122	Generator	Diesel 80KW	650.0
123	Concrete Pump	Putzmeister BSA 1407	2000.0
124	Mini Excavator	Kubota U55-4	700.0
125	Scissor Lift	JLG 3246ES	

New Booking

Equipment ID:
Start Date:
End Date:
City:
Zip Code:

Confirm Booking

Message

Booking created successfully!
Total price: 27900.0 SAR

OK

My Rentals

Cancel Booking

My Payments

Update My Info

EJAR – Customer Portal

Welcome, jahiz3

Mobile: 0512345678 | Email: jahiz3@gmail.com

Available Equipments

Equip ID	Type	Model	Price
----------	------	-------	-------

New Booking

Equipment ID:
Start Date:
End Date:
City:
Zip Code:

Confirm Booking

My Rentals

Cancel Booking

My Payments

Update My Info

Logout

Customer Log In:

EAJAR - Login



شركة بدائل الإيجار للمعدات المحدودة
Machinery Rental Alternatives Co. Ltd.

Welcome to EJAR

☒ Customer ☐ Employee

Customer name:

Password:

Hint Password: last 4 digits of your mobile number.

As for the password hint:
customers use their phone number to
log in as this attribute is kept private for
the customer.

EAJAR - Customer Portal

Welcome, AlJazeera
Mobile: 0555783000 | Email: aljazeera@gmail.com

Available Equipments

Equip ID	Type	Model	Price
123	Concrete Pump	Putzmeister BSA 1407	2000.0
124	Mini Excavator	Kubota U55-4	700.0
125	Scissor Lift	JLG 3246ES	550.0

New Booking

Equipment ID:

Start Date:

End Date:

City:

Zip Code:

Logged in users can log out and
return to the main page
smoothly, as their rentals exist
already.

My Rentals

Rental ID	Start	End	City	Zip
2004	2024-12-05	2024-12-12	Dammam	33777

My Payments

Invoice	Pay Date	Total Price
5004	2024-12-12	2800.0

Customers can view their own records of
rentals and payments, they can also
update their info

Update My Info

☒ ?

New mobile:

New email:

Employee:

EJAR - Login

EJAR

شركة بدائل الإيجار للمعدات المحدودة

Machinery Rental Alternatives Co. Ltd.

Welcome to EJAR

Customer

Employee

Employee ID:

Password:

Hint: Use your default company password.

LoginRegister

Employees begin by either registering or logging in assuming password is known for them by company’s policy, default password would be: 1234

Newly registered Employees are also accepted

Employee registration

?

Name:

OK

Cancel

After successful log-in /register:

EJAR - Employee Portal

EJAR – Employee Dashboard

ID: 1 | Ahmed Al-Saif

Equipment I Currently Manage

Equip ID	Type	Model	Price	Status
Choose equipment to manage				
Stop managing				
Update / Delete				

Rentals

Rental ID	Customer	Email	Start	End	Equip ID	Total Pri...	Emp ID
2004	AlJazeera	aljazeer...	2024-12...	2024-12...	113	2800.0	1
2002	BuildRight	building...	2024-11...	2024-11...	111	1500.0	1
2011	BuildRight	building...	2024-12...	2024-12...	120	3000.0	4
2003	EliteBuil...	elitebul...	2024-12...	2024-12...	112	3600.0	3
2012	EliteBuil...	elitebul...	2024-12...	2024-12...	121	900.0	2
2010	FutureC...	futureco...	2024-12...	2024-12...	119	4800.0	3
2013	Jahez	Jahez@...	2025-11...	2026-02...	122	40950.0	
2014	Jahez	Jahez@...	2025-11...	2026-02...	122	40950.0	
2006	MegaPr...	megapr...	2024-12...	2024-12...	115	1300.0	3
2001	Perfect ...	perfecta...	2024-11...	2024-11...	110	3750.0	2
2008	PrimeCo...	primeco...	2024-12...	2024-12...	117	1200.0	5
2007	Skyline...	skylinec...	2024-12...	2024-12...	116	1050.0	2

Supervise a rental

Rentals I supervise

View Customer Contact

Add New Equipment

All Equipments

All Payments

Equipments above price

Delete My Record

Logout

Employees can View all rentals and basic customer’s info except for phone number. But not payments exact details.

When Employee adds equipment **they** become it's manager by default.

After successful log-in /register:

EJAR – Employee Portal

EJAR – Employee Dashboard

Minimize ID: 7 | sara

Equipment I Currently Manage

Equip ID	Type	Model	Price	Status
Choose equipment to manage				
Stop managing				
Update / Delete				

Rentals

Rental ID	Customer	Email	Start	End	Equip ID	Total P...	Emp ID	Emp N...
2004	AlJazeera	aljaze...	2024-1...	2024-1...	113	2800.0	1	Ahmed ...
2002	BuildRl...	buildrig...	2024-1...	2024-1...	111	1500.0	1	Ahmed ...
2011	BuildRl...	buildrig...	2024-1...	2024-1...	120	3000.0	4	Khalid ...
2003	EliteBul...	elitebul...	2024-1...	2024-1...	112	3600.0	3	Noura ...
2012	EliteBul...	elitebul...	2024-1...	2024-1...	121	900.0	2	Sara Al...
2010	Future...	futurec...	2024-1...	2024-1...	119	4800.0	3	Noura ...
2006	MegaP...	megapr...	2024-1...	2024-1...	115	1300.0	3	Noura ...
2001	Perfect...	perfect...	2024-1...	2024-1...	110	3750.0	2	Sara Al...
2008	PrimeC...	primec...	2024-1...	2024-1...	117	1200.0	5	Reem ...
2007	Skyline...	skyline...	2024-1...	2024-1...	116	1050.0	2	Sara Al...
2009	SolidGr...	solidgr...	2024-1...	2024-1...	118	6300.0	6	Omar A...
2005	Strong...	strongb...	2024-1...	2024-1...	114	6000.0	4	Khalid ...

Supervise a rental

Rentals I supervise

View Customer Contact

Add New Equipment

All Equipments

All Payments

Equipments above price

Sales by type

Delete My Record

Logout

Add New Equipment

Type:

Model:

Price (per day):

Employees can manage one equipment at a moment to preform update/delete operations on their managed equipment:

Employees are allowed to remove their record **only if not supervising any rentals**

Employees can manage many rentals and that gives them the benefit of viewing customers phone number

Connection Code:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL =
"jdbc:mysql://localhost:3306/ejar?useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASS = ""; //use system password

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

EjarLogin :

- This code opens a database connection and prepares an SQL query to retrieve the mobile number of a customer based on the username entered during login.

```
private void handleCustomerLogin(String customerName, String password) {  
  
    try (Connection conn = DBConnection.getConnection());  
        PreparedStatement ps = conn.prepareStatement(  
            "SELECT Cus_mobile FROM CUSTOMER WHERE Cus_name = ?")) {
```

- This code validates that the entered employee ID is a number, and then connects to the database to retrieve the employee's name using that ID

```
private void handleEmployeeLogin(String id, String password) {  
  
    int empId;  
    try {  
        empId = Integer.parseInt(id);  
    } catch (NumberFormatException ex) {  
        JOptionPane.showMessageDialog(this,  
            "Employee ID must be a number.",  
            "Error",  
            JOptionPane.WARNING_MESSAGE);  
        return;  
    }  
  
    try (Connection conn = DBConnection.getConnection());  
        PreparedStatement ps = conn.prepareStatement(  
            "SELECT Emp_name FROM EMPLOYEE WHERE Emp_id = ?")) {
```

- This code opens a database connection and inserts a new customer record into the CUSTOMER table by storing the name, mobile number, and email entered during registration

```
private void registerCustomer() {  
  
    try (Connection conn = DBConnection.getConnection());  
        PreparedStatement ps = conn.prepareStatement(  
            "INSERT INTO CUSTOMER (Cus_name, Cus_mobile, Email) VALUES (?, ?, ?)")) {
```

- This code displays a form to collect a new employee's ID and name, and then inserts these values into the EMPLOYEE table to register the employee in the database.

```
private void registerEmployee() {

    JTextField idField = new JTextField();
    JTextField nameField = new JTextField();

    JPanel panel = new JPanel(new GridLayout(2, 2, 5, 5));
    panel.add(new JLabel("Employee ID:"));
    panel.add(idField);
    panel.add(new JLabel("Name:"));
    panel.add(nameField);

    int result = JOptionPane.showConfirmDialog(this, panel,
        "Employee registration", JOptionPane.OK_CANCEL_OPTION);

    if (result != JOptionPane.OK_OPTION) return;

    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(
            "INSERT INTO EMPLOYEE (Emp_id, Emp_name) VALUES (?, ?)")) {
```

CustomerPanel :

- This method collects the customer's name, mobile number, and email from the form, validates the input, and then inserts the new customer record into the CUSTOMER table using a prepared SQL statement

```
private void addCustomer() {
    String name = txtName.getText().trim();
    String mobile = txtMobile.getText().trim();
    String email = txtEmail.getText().trim();

    if (name.isEmpty()) {
        msg("Customer Name is required (Primary Key).");
        return;
    }

    final String sql = "INSERT INTO CUSTOMER (Cus_name, Cus_mobile, Email) VALUES (?, ?, ?)";

    try (Connection con = DBConnection.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
```

- This method reloads the customer table by clearing the current rows and executing a SQL query that retrieves all customers' names, mobile numbers, and emails, ordered alphabetically.

```
private void loadData() {
    model.setRowCount(0);

    final String sql
        = "SELECT Cus_name, Cus_mobile, Email FROM CUSTOMER ORDER BY Cus_name";

    try (Connection con = DBConnection.getConnection(); PreparedStatement ps = con.prepareStatement(sql); Re
```

PaymentPanel :

- This method collects the total amount and rental ID from the form, while the system automatically generates the invoice number and current payment date (based on confirmed booking) , and then inserts a new payment record into the PAYMENT table using a prepared SQL INSERT statement

```
private void addPayment(){
try {
int inv = Integer.parseInt(txtInv.getText().trim());
double total = Double.parseDouble(txtTotal.getText().trim());
int rental = Integer.parseInt(txtRental.getText().trim());

String sql = "INSERT INTO PAYMENT VALUES (?, ?, ?, ?)";

try(Connection c = DBConnection.getConnection();
PreparedStatement ps = c.prepareStatement(sql)){
```

- This code updates an existing payment record by modifying the payment date, total price, and associated rental ID using a prepared SQL UPDATE statement that targets the specified invoice number.

```
private void updatePayment(){
try {
String sql =
"UPDATE PAYMENT SET Pay_date=?, Total_price=?, P_rental=? WHERE Invoice=?";

try(Connection c = DBConnection.getConnection();
PreparedStatement ps = c.prepareStatement(sql)){
```

- This method refreshes the payment table by clearing existing rows and executing a SQL query that retrieves all payment records ordered by their invoice number

```
private void loadData(){
    model.setRowCount(0);

    try(Connection c = DBConnection.getConnection();
        Statement st = c.createStatement();
        ResultSet rs = st.executeQuery("SELECT * FROM PAYMENT ORDER BY Invoice")){
```

Customer Dashboard:

```
public CustomerDashboard(String customerName, boolean newlyRegistered) {
```

```
    try {
        conn = DBConnection.getConnection();
        loadAvailableFromDB();
        loadCustomerInfoFromDB();
    } catch (Exception ex) {
        ex.printStackTrace();
        JOptionPane.showMessageDialog(this, "DB connection error");
    }

    if (this.newlyRegistered) {
        JOptionPane.showMessageDialog(
            this,
            "Welcome, " + customerName +
            "\nYou must create at least one rental now.",
            "EJAR Notice",
            JOptionPane.INFORMATION_MESSAGE
        );
    }
}
```

- This constructor initializes the Customer Dashboard by establishing a database connection, loading customer and rental data, and displaying a welcome notice if the user is newly registered

```
private void performCancelBooking() {
    String checkSql = "SELECT Equ_id FROM MAKES WHERE Rent_id = ? AND C_name = ?";
    "UPDATE PAYMENT SET Total_price = 0 WHERE P_rental = ?")) {
    "DELETE FROM MAKES WHERE Rent_id = ?")) {
```

- The performCancelBooking() method allows a customer to cancel one of their rentals. It first verifies that the rental belongs to the logged-in customer by checking the MAKES table using the rental ID and customer name. If the rental is valid, the method updates the PAYMENT table by setting the total price to zero to indicate cancellation, and then deletes the corresponding record from the MAKES table to remove the link between the customer and the rental.

```

private void loadAvailableFromDB() {

String sql =
    "SELECT Equip_id, Type, Model, Price " +
    "FROM EQUIPMENT1 " +
    "WHERE Status = 'available'";

try (PreparedStatement ps = conn.prepareStatement(sql);      +
    ResultSet rs = ps.executeQuery()) {

    "WHERE Cus_name = ?";
}

```

- This code updates an existing customer's mobile number, and email in the CUSTOMER table by executing an SQL UPDATE statement that matches the original customer name.
- This query retrieves all available equipment by selecting the equipment ID, type, model, and price from the EQUIPMENT1 view, filtering only rows where the status is marked as 'available'

Rental Panel:

- This method adds a new rental record by reading input fields, validating and converting their values, and inserting the rental information into the RENTAL table in the database using a prepared statement

```

private void addRental(){
try {
int id = Integer.parseInt(txtRentalId.getText().trim());
double total = Double.parseDouble(txtTotal.getText().trim());
int equip = Integer.parseInt(txtEquipId.getText().trim());
int emp = Integer.parseInt(txtEmpId.getText().trim());

String sql =
    "INSERT INTO RENTAL VALUES (?, ?, ?, ?, ?, ?, ?, ?)";

try(Connection c = DBConnection.getConnection();
    PreparedStatement ps = c.prepareStatement(sql)){

```

- This method loads all rental records from the database, ordered by their ID, and populates the table model with the retrieved data.

```

private void loadData(){
model.setRowCount(0);

try(Connection c = DBConnection.getConnection();
    Statement st = c.createStatement();
    ResultSet rs = st.executeQuery("SELECT * FROM RENTAL1 ORDER BY Rental_id")){

    while(rs.next()){
        Vector<Object> row = new Vector<>();

```

EmployeePanel :

- This code checks whether the employee is supervising any rentals by counting all RENTAL records linked to the employee's ID. If the count is greater than zero, the system prevents deletion to maintain referential integrity and avoid orphaned foreign-key relationships.

```
// Optional safety: prevent delete if supervising rentals
try (Connection c = DBConnection.getConnection();
    PreparedStatement ps = c.prepareStatement(
        "SELECT COUNT(*) AS cnt FROM RENTAL WHERE Employee_id = ?")) {
```

- This code gets all employees from the database by running a SELECT query that returns their ID, name, and equipment ID, and it sorts them by employee ID so the list appears organized.

```
try(Connection c = DBConnection.getConnection();
    Statement st = c.createStatement();
    ResultSet rs = st.executeQuery("SELECT Emp_id, Emp_name, Eq_id FROM EMPLOYEE ORDER BY Emp_id")){
```

- This code safely deletes an employee from the database by using a prepared statement, which prevents SQL errors and ensures that only the employee with the matching ID gets removed.

```
try(Connection c = DBConnection.getConnection();
    PreparedStatement ps = c.prepareStatement("DELETE FROM EMPLOYEE WHERE Emp_id=?")){
```

- This code updates an existing employee's record by modifying their name and assigned equipment using UPDATE statement.

```
String sql = "UPDATE EMPLOYEE SET Emp_name = ?, Eq_id = ? WHERE Emp_id = ?";
```

```
try(Connection c = DBConnection.getConnection();
```

- This code inserts a new employee into the EMPLOYEE table by adding the ID, name, and assigned equipment.

```
String sql = "INSERT INTO EMPLOYEE (Emp_id, Emp_name, Eq_id) VALUES (?, ?, ?)";
```

```
try(Connection c = DBConnection.getConnection());
```

- This code checks whether an equipment item exists in the EQUIPMENT table by counting how many records match the given ID.

```
String sql = "SELECT COUNT(*) AS cnt FROM EQUIPMENT WHERE Equip_id = ?";
```

```
try (Connection c = DBConnection.getConnection());
```

EquipmentPanel :

- This method allows the user to add a new equipment record to the database. It collects input from text fields for equipment ID, type, model, price, and status. It validates that the required fields are not empty and that the ID and price are numeric. If the status is not provided, it defaults to "Available." After validation, the method prepares an SQL INSERT statement to store the new equipment in the database.

```
private void addEquipment(){

    String idText = txtId.getText().trim();
    String type = txtType.getText().trim();
    String modelTxt = txtModel.getText().trim();
    String priceTxt = txtPrice.getText().trim();
    String status = txtStatus.getText().trim();

    if (idText.isEmpty() || type.isEmpty() || modelTxt.isEmpty() || priceTxt.isEmpty()) {
        msg("Equip_id, Type, Model, Price are required.");
        return;
    }

    int id;
    double price;
    try {
        id = Integer.parseInt(idText);
        price = Double.parseDouble(priceTxt);
    } catch (NumberFormatException ex) {
        msg("Equip_id and Price must be numeric.");
        return;
    }

    if (status.isEmpty()) status = "Available";

    final String sql = "INSERT INTO EQUIPMENT (Equip_id, Type, Model, Price) VALUES (?, ?, ?, ?)";
```

- The `updateEquipment()` method updates an existing equipment record in the database. It first checks if the user has selected a row in the table. Then it reads the values from the text fields for equipment ID, type, model, price, and status. It validates that the ID is not empty and that the ID and price are numeric. After validation, it prepares an SQL UPDATE statement to modify the corresponding equipment record in the database with the new values.

```
private void updateEquipment(){

    int row = table.getSelectedRow();
    if (row == -1){
        msg("Select a row to update.");
        return;
    }

    String idText = txtId.getText().trim();
    String type = txtType.getText().trim();
    String modelTxt = txtModel.getText().trim();
    String priceTxt = txtPrice.getText().trim();
    String status = txtStatus.getText().trim();

    if (idText.isEmpty()) {
        msg("Equip_id is required.");
        return;
    }

    int id;
    double price;
    try {
        id = Integer.parseInt(idText);
        price = Double.parseDouble(priceTxt);
    } catch (NumberFormatException ex) {
        msg("Equip_id and Price must be numeric.");
        return;
    }

    final String sql = "UPDATE EQUIPMENT SET Type=?, Model=?, Price=? WHERE Equip_id=?";
```

- This method allows only an authorized employee to delete equipment by checking that a row is selected, confirming the deletion with the user, and then executing an SQL DELETE statement to remove the equipment with the specified ID; after deletion, the table is refreshed and the input fields are cleared

```
private void deleteEquipment(){

    int row = table.getSelectedRow();
    if (row == -1){
        msg("Select a row to delete.");
        return;
    }

    int id = Integer.parseInt(model.getValueAt(row,0).toString());

    if (JOptionPane.showConfirmDialog(this,
        "Delete equipment " + id + " ?",
        "Confirm delete", JOptionPane.YES_NO_OPTION) != JOptionPane.YES_OPTION)
        return;

    final String sql = "DELETE FROM EQUIPMENT WHERE Equip_id=?";
```

- This method retrieves all equipment records from the database and displays them in the table. It first clears any existing rows in the table model. Then it executes an SQL SELECT query to get the equipment ID, type, model, price, and status from the database, ordered by ID. For each record, it creates a row and adds it to the table model. If a database error occurs, it shows an error message.

```
private void loadData() {

    model.setRowCount(0);

    final String sql = "SELECT Equip_id, Type, Model, Price, Status FROM EQUIPMENT1 ORDER BY Equip_id"
```

MakesPanel :

- This method retrieves all records from the MAKES table, which represents relationships between customers, equipment, and rentals, and displays them in the table. It first clears any existing rows in the table model. Then it executes an SQL SELECT query to get the customer name, equipment ID, and rental ID, ordered by customer name, equipment ID, and rental ID. For each record, it creates a row and adds it to the table model. If an error occurs, it shows an error message.

```
private void loadData(){
    model.setRowCount(0);

    final String sql = "SELECT C_name, Equ_id, Rent_id FROM MAKES ORDER BY C_name, Equ_id, Rent_id";

    try(Connection c = DBConnection.getConnection();
        PreparedStatement ps = c.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()){

        while(rs.next()){
            Vector<Object> row = new Vector<>();
            row.add(rs.getString("C_name"));
            row.add(rs.getInt("Equ_id"));
            row.add(rs.getInt("Rent_id"));
            model.addRow(row);
        }

    } catch(Exception ex){ error(ex); }
```

- This method adds a new record to the MAKES table, which links a customer with equipment and a rental. It first reads the values from the input fields for customer name, equipment ID, and rental ID. It checks that none of the fields are empty and that the equipment ID and rental ID are numeric. After validation, it connects to the database and prepares to insert the new record.

```
private void addMakes(){
try (Connection c = DBConnection.getConnection()) {

    if (!exists(c, "SELECT 1 FROM CUSTOMER WHERE Cus_name = ?", nameText)) {
        msg("C_name not exist in CUSTOMER.");
        return;
    }

    if (!exists(c, "SELECT 1 FROM EQUIPMENT WHERE Equ_id = ?", equipId)) {
        msg("Equ_id not exist in EQUIPMENT.");
        return;
    }

    if (!exists(c, "SELECT 1 FROM RENTAL WHERE Rental_id = ?", rentId)) {
        msg("Rent_id not exist in RENTAL.");
        return;
    }

}
```

EmployeeDashboard:

- The constructor creates the employee dashboard window. It stores the employee's ID and name, sets the window title, size, and position, and adds a background image. It builds the header, main content area, and bottom bar. Then it connects to the database and loads the equipment managed by the employee and all rental records. If a database error occurs, it shows an error message.

```
private void showAllEquipments() {  
    String sql = "SELECT Equip_id, Type, Model, Price, Status FROM equipment1";
```

```
public EmployeeDashboard(int employeeId, String employeeName) {  
    this.employeeId = employeeId;  
    this.employeeName = employeeName;  
  
    setTitle("EJAR - Employee Portal");  
    setSize(1200, 700);  
    setLocationRelativeTo(null);  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
  
    //String bgPath = "C:\\Users\\user\\OneDrive\\Documents\\380CSC Database\\CSC380_Phase-3-_  
    BackgroundPanel bg = new BackgroundPanel("/images/dashboard.jpg");  
    setContentPane(bg);  
    bg.setLayout(new BorderLayout());  
  
    add(buildHeader(), BorderLayout.NORTH);  
    add(buildCenterArea(), BorderLayout.CENTER);  
    add(buildBottomBar(), BorderLayout.SOUTH);  
  
    try {  
        this.conn = DBConnection.getConnection();  
        loadManagedEquipment();  
        loadAllRentals();  
    } catch (Exception ex) {  
        ex.printStackTrace();  
        JOptionPane.showMessageDialog(  

```

- This query calculates total sales for each equipment type by joining the EQUIPMENT, MAKES, RENTAL, and PAYMENT tables, summing all payments related to rentals of that type, grouping the results by equipment type, and ordering them from the highest total sales to the lowest

```
//          viewTypeSales() calculates total sales per machinery type  
private void viewTypeSales() {  
    String sql =  
        "SELECT E.Type, SUM(P.Total_price) AS Total_sales " +  
        "FROM EQUIPMENT E " +  
        "JOIN MAKES M ON E.Equip_id = M.Equ_id " +  
        "JOIN RENTAL R ON M.Rent_id = R.Rental_id " +  
        "JOIN PAYMENT P ON R.Rental_id = P.P_rental " +  
        "GROUP BY E.Type " +  
        "ORDER BY Total_sales DESC";  
  
    try (PreparedStatement ps = conn.prepareStatement(sql);  
        ResultSet rs = ps.executeQuery()) {
```

- This code builds an SQL query that selects each equipment's ID, type, model, and price from the EQUIPMENT table, but only for rows whose price is greater than a user-entered minimum, and orders the results from highest price to lowest using a prepared statement parameter for that minimum price

```
private void viewEquipmentsAbovePrice() {
    String sql =
        "SELECT Equip_id, Type, Model, Price " +
        "FROM EQUIPMENT " +
        "WHERE Price > ? " +
        "ORDER BY Price DESC";

    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setDouble(1, minPrice);
    }
}
```

- This query loads all rental records by joining the RENTAL, MAKES, CUSTOMER, and PAYMENT tables to retrieve each rental's ID, customer name, email, rental dates, equipment ID, total price, and assigned employee, ensuring that rentals without payment records still appear using a LEFT JOIN.

```
private void loadAllRentals() {
    String sql =
        "SELECT R.Rental_id, " +
        "       C.Cus_name AS Customer, " +
        "       C.Email    AS Email, " +
        "       R.Start_date, " +
        "       R.End_date, " +
        "       M.Equ_id   AS Equip_id, " +
        "       P.Total_price, " +
        "       R.Employee_id " +
        "FROM RENTAL R " +
        "JOIN MAKES M   ON R.Rental_id = M.Rent_id " +
        "JOIN CUSTOMER C ON M.C_name    = C.Cus_name " +
        "LEFT JOIN PAYMENT P ON P.P_rental = R.Rental_id";
}
```

- This query retrieves all rentals supervised by the logged-in employee by selecting each rental's ID, start and end dates, city, zip code, and total payment amount, joining the RENTAL and PAYMENT tables, and filtering the results using the employee's ID

```
private void showRentalsISupervise() {
    String sql =
        "SELECT R.Rental_id, R.Start_date, R.End_date, " +
        "       R.City, R.Zip_code, P.Total_price " +
        "FROM RENTAL R " +
        "JOIN PAYMENT P ON P.P_rental = R.Rental_id " +
        "WHERE R.Employee_id = ?";
}
```